

Guía de Ejercicios Prácticos: Punteros en C

Ejercicio 1: Calibración de Niveles de Agua

Enunciado: En el sistema de riego por goteo automatizado de una finca olivícola en Vichigasta, la lectura del **nivel de agua** de un tanque (variable `float`) a veces requiere una **corrección en tiempo real**.

Escriba una función `calibrar_nivel(float *nivel_actual_ptr, float error_calibracion)` que reciba:

1. La **dirección** del nivel actual del tanque.
2. Un valor `float` que representa el error que debe restarse a la lectura.

La función debe **modificar el valor original** del nivel, restándole el `error_calibracion` directamente a través del puntero.

Requerimiento:

- Declare el nivel inicial del tanque en el `main` como 95.5 litros.
- Use un error de calibración de 2.1 litros.
- Imprima el nivel antes y después de la llamada a la función.

Ejercicio 2: Inventario de Productos Regionales

Enunciado: Un sistema de inventario para un almacén de dulces regionales (membrillo, nueces, vino) usa un array para rastrear la **cantidad de stock** disponible. Queremos implementar un proceso rápido para **duplicar el inventario** de un producto específico.

Escriba una función `duplicar_stock(int *stock_ptr)` que reciba la **dirección de memoria** de una variable entera y **multiplique su valor por 2** usando el operador de desreferencia (*).

Requerimiento:

- En el `main`, declare un array `int stock_bodega[3] = {150, 80, 20};` (Vino, Dulces, Aceite).
- Llame a `duplicar_stock` pasando la **dirección del segundo elemento** del array (el stock de dulces).
- Imprima todo el array antes y después de la llamada para verificar que solo el segundo elemento cambió.

Ejercicio 3: Cálculo de Presupuesto del Proyecto (Ing. en Sistemas)

Guía de Ejercicios Prácticos: Punteros en C

Enunciado: En la gestión de proyectos de desarrollo de software, es común calcular rápidamente el **costo total** a partir de variables de costo unitario y horas. La función debe modificar una variable de resultado externa.

Escriba una función `calcular_costo(int horas, float tarifa, float *costo_total_ptr)` que:

1. Reciba las `horas` (entero) y la `tarifa` (flotante) por valor.
2. Reciba la **dirección** de la variable `costo_total` (flotante) que está en el `main`.
3. Calcule el producto (`horas * tarifa`) y **almacene el resultado** en la dirección de memoria proporcionada.

Requerimiento:

- En el `main`, use `horas = 40` y `tarifa = 550.75` (pesos/hora).
- Declare `float costo_final = 0.0;`
- Pase la dirección de `costo_final` a la función y luego imprima su nuevo valor.

Ejercicio 4: Verificación de Estado Operativo (Punteros y Punteros a Punteros)

Enunciado: Un sistema de supervisión de maquinaria crítica en una planta industrial necesita que el estado de la máquina se pueda reportar y cambiar desde distintos módulos del programa. Usaremos **punteros a punteros** para la máxima flexibilidad.

1. Declare una variable entera `int estado_maquina = 0;` (0: Apagada).
2. Declare un puntero simple a entero `int *p_estado = &estado_maquina;`
3. Declare un **puntero a puntero** a entero `int **pp_estado = &p_estado;`

Escriba una función `encender_maquina(int **estado_doble_ptr)` que reciba el puntero a puntero y **cambie el valor original** de `estado_maquina` a 1 (Encendida) utilizando solo la variable `estado_doble_ptr`.

Requerimiento:

- En el `main`, imprima el valor de `estado_maquina` (el valor original) antes y después de llamar a `encender_maquina(&p_estado)`.

Pista:

El valor se accede con el triple asterisco (`*** estado_doble_ptr`) o, más limpiamente, con el doble asterisco (`** estado_doble_ptr`) para acceder al puntero, y luego a su valor.